Data Preparation

Date	01-11-2026
Team ID	EN25CS3011097
Project Name	Global Olympic Sports Performance Analysis

Data preparation refers to the process of cleaning, standardizing, transforming, and organizing data so that it is suitable for analysis and reporting. In a Snowflake project, data preparation involves working with the ingested RAW data, identifying data quality issues, creating STAGING tables, applying required transformations, and preparing reliable data for the CURATED and MART layers. For this project the RAW table is **OLYMPIC_DB.SPORTS.OLYMPIC_EVENTS** (8,988 rows × 56 columns) loaded from *athelete_events.csv* (Olympic data 1992–2026).

Activity 1: Identifying and Handling Null Values

The dataset holds four record types in one table (*dataset_type*): athletes, medal_results, alltime_country_rankings and games_summary. Each record type fills only its own columns, so the other columns are blank (NULL). These NULLs are structural, not errors, so they are handled by filtering on *dataset_type* rather than by deleting rows or filling values.

Code

```
-- 1. Row count of every record type (RAW layer)
SELECT dataset_type, COUNT(*) AS row_count
FROM OLYMPIC_DB.SPORTS.OLYMPIC_EVENTS
GROUP BY dataset_type
ORDER BY row_count DESC;

-- 2. Profile NULLs in the key columns of each record type
SELECT dataset_type,
       COUNT(*)                              AS total_rows,
       COUNT_IF(country IS NULL)             AS null_country,
       COUNT_IF(athlete_id IS NULL)          AS null_athlete_id,
       COUNT_IF(total_medals IS NULL)        AS null_total_medals,
       COUNT_IF(total_medal_points IS NULL)  AS null_total_medal_points
FROM OLYMPIC_DB.SPORTS.OLYMPIC_EVENTS
GROUP BY dataset_type;

-- 3. Handle NULLs: one STAGING view per record type (no rows deleted, no values invented)
CREATE OR REPLACE VIEW STG_ATHLETES AS
  SELECT * FROM OLYMPIC_EVENTS WHERE dataset_type = 'athletes';
CREATE OR REPLACE VIEW STG_MEDAL_RESULTS AS
  SELECT * FROM OLYMPIC_EVENTS WHERE dataset_type = 'medal_results';
CREATE OR REPLACE VIEW STG_COUNTRY_RANKINGS AS
  SELECT * FROM OLYMPIC_EVENTS WHERE dataset_type = 'alltime_country_rankings';
CREATE OR REPLACE VIEW STG_GAMES_SUMMARY AS
  SELECT * FROM OLYMPIC_EVENTS WHERE dataset_type = 'games_summary';
```

Output

Row count by record type (query 1) – matches the CSV file (8,000 + 910 + 60 + 18 = 8,988):

DATASET_TYPE	ROW_COUNT
athletes	8,000
medal_results	910
alltime_country_rankings	60
games_summary	18
Total	**8,988**

Key columns populated by each record type (the other columns are NULL by design):

dataset_type	Populated key columns
athletes	athlete_id, name, gender, age_at_peak, sport, height_cm, weight_kg, bmi
medal_results	year, host_city, country, gold, silver, bronze, total_medals, medal_points
alltime_country_rankings	country, total_gold, total_silver, total_bronze, total_medal_points, overall_rank
games_summary	year, host_city, total_athletes, participating_nations, top_country

Result: NULLs are structural, so no rows were deleted or imputed. Four STAGING views isolate each record type and return 8,000 / 910 / 60 / 18 rows respectively.

Activity 2: Duplicate Record Handling

Duplicates were checked with an explicit rule: an athlete record is a duplicate only when the same *athlete_id* appears more than once, and a medal record only when the same country and year appear more than once. Repeated athlete **names** are not treated as duplicates because they belong to different *athlete_id* values (different athletes). Athlete counts therefore always use COUNT(DISTINCT athlete_id).

Code

```
-- 1. Duplicate check on the athlete key (athlete_id)
SELECT athlete_id, COUNT(*) AS records
FROM STG_ATHLETES
GROUP BY athlete_id
HAVING COUNT(*) > 1;

-- 2. Duplicate check on the medal key (country + year)
SELECT country, year, COUNT(*) AS records
FROM STG_MEDAL_RESULTS
GROUP BY country, year
HAVING COUNT(*) > 1;

-- 3. Repeated names: are they genuine different athletes?
SELECT name, COUNT(DISTINCT athlete_id) AS events
FROM STG_ATHLETES
GROUP BY name
ORDER BY events DESC, name
LIMIT 10;
```

Output

Query 3 – the most repeated names (first 5 of 10 rows). Each name maps to several different athlete_id values, so these are genuine separate athletes and are kept:

NAME	EVENTS (distinct athlete_id)
Wei Ngozi	17
Wei Ferreira	15
Wei Hernández	14
Wei Rossi	14
Yuki Johnson	14

Result: duplicates are defined on athlete_id and country + year, not on name. No records were removed, so the row count stays 8,988 and athlete counts use COUNT(DISTINCT athlete_id) to avoid double counting.

Activity 3: Data Cleaning and Standardization

Cleaning and standardization were applied while loading and in a cleaned view. The CSV file format treats blank and the text NULL as real NULL values and skips the header row. Text columns are trimmed, categorical text is standardized to one format, the year is cast to a whole number and numeric fields stay numeric measures (text fields are dimensions). Country is later given the geographic role Country/Region in Tableau.

Code

```
-- 1. File format used when loading (blank / 'NULL' text become NULL)
CREATE OR REPLACE FILE FORMAT CSV_FF
  TYPE = CSV FIELD_DELIMITER = ',' SKIP_HEADER = 1
  FIELD_OPTIONALLY_ENCLOSED_BY = '"' NULL_IF = ('', 'NULL');

-- 2. Cleaned and standardized view over the RAW table
CREATE OR REPLACE VIEW CLN_OLYMPIC_EVENTS AS
SELECT dataset_type,
       CAST(year AS INTEGER)          AS year,
       TRIM(host_city)                AS host_city,
       TRIM(host_country)             AS host_country,
       INITCAP(TRIM(season))          AS season,
       TRIM(country)                  AS country,
       UPPER(TRIM(noc_code))          AS noc_code,
       TRIM(region)                   AS region,
       athlete_id,
       TRIM(name)                     AS name,
       INITCAP(TRIM(gender))          AS gender,
       TRIM(sport)                    AS sport,
       gold, silver, bronze, total_medals, medal_points,
       total_gold, total_silver, total_bronze, total_medal_points,
       age_at_peak, height_cm, weight_kg, bmi
FROM OLYMPIC_EVENTS;

-- 3. Validate: the cleaning step must not lose or add rows
SELECT COUNT(*) AS rows_in_cleaned_view FROM CLN_OLYMPIC_EVENTS;
```

Output

Check	Result
Rows in RAW table OLYMPIC_EVENTS	8,988
Rows in CLN_OLYMPIC_EVENTS	8,988
Columns in RAW table	56
Record types preserved	athletes, medal_results, alltime_country_rankings, games_summary

Result: text values are trimmed and standardized, year is a whole number, and the cleaned view keeps every record (8,988).

Activity 4: Data Transformation and Validation

The cleaned data was transformed for analysis. A derived field Total Medals Calculated (gold + silver + bronze) was created, Top-10 ranking rules were applied with ORDER BY and LIMIT, and MART views were created for the dashboard. The results were validated by reconciling the medal totals with the gold, silver and bronze counts.

Code

```
-- 1. Derived field and MART view: medals by country
CREATE OR REPLACE VIEW MART_COUNTRY_MEDALS AS
SELECT country,
       SUM(gold)   AS gold,
       SUM(silver) AS silver,
       SUM(bronze) AS bronze,
       SUM(total_medals) AS total_medals,
       SUM(gold + silver + bronze) AS total_medals_calculated
FROM CLN_OLYMPIC_EVENTS
WHERE dataset_type = 'medal_results'
GROUP BY country;

-- 2. MART view: athletes per sport
CREATE OR REPLACE VIEW MART_SPORT_PARTICIPATION AS
SELECT sport, COUNT(DISTINCT athlete_id) AS athletes
FROM CLN_OLYMPIC_EVENTS
WHERE dataset_type = 'athletes'
GROUP BY sport;

-- 3. Validation: reported total must equal gold + silver + bronze
SELECT country, gold, silver, bronze, total_medals,
       total_medals_calculated,
       IFF(total_medals = total_medals_calculated, 'PASS', 'FAIL') AS check_result
FROM MART_COUNTRY_MEDALS
ORDER BY total_medals DESC
LIMIT 10;
```

Output

Validation of the first 5 of 10 rows (total medals = gold + silver + bronze):

COUNTRY	GOLD	SILVER	BRONZE	TOTAL_MEDALS	CALCULATED	CHECK
United States	250	208	277	735	735	PASS
China	230	230	236	696	696	PASS
Russia	130	150	149	429	429	PASS
Great Britain	120	138	143	401	401	PASS
Germany	116	103	117	336	336	PASS

Result: the transformed MART data is consistent with the source (for example United States 250 + 208 + 277 = 735) and is ready for the Tableau worksheets, dashboard and story.